慢在哪，以及先动哪

1,528 次调用 + 现场 8 轮实测 · 2026-11-01 · 芝士

一、每步各花多少时间

取最近 8 轮真实会话，逐轮拆开：

轮次总时长	调用次数	每次调用平均	首字出现	请求前准备
22.6s	6	3.8s	3.31s	1.79s
22.2s	7	3.2s	1.01s	0.07s
72.6s	10	7.3s	6.15s	1.87s
136.1s	20	6.8s	0.96s	0.07s
277.2s	29	9.6s	3.36s	0.35s
618.5s	79	7.8s	1.31s	0.07s
814.0s	119	6.8s	8.57s	0.12s
1011.3s	58	17.4s	2.52s	0.07s

合计 2,975 秒、328 次调用，平均 9.1 秒一次，中位 7.0 秒。
拆得最细的一轮（22.6 秒、6 次调用）：模型等待 **11.5 秒**、工具执行 **5.9 秒**、开工准备 **3.3 秒**、收尾 **1.8 秒**。

二、每次调用自己的首字延迟

新字段生效后，直接读到 14 次调用各自的首字延迟：
中位 1.73 秒，均值 2.64 秒，范围 1.31–8.83 秒。9 次在 2 秒内，13 次在 5 秒内。
也就是说：**人坐在那里等，多数时候不到 2 秒就能看到第一个字**；慢的不是「等第一句话」，是一轮里这样的调用有几十上百次。

三、三个之前不知道的

长尾吃掉四分之一：超过 30 秒的调用只占 **3.3%**，但吃掉全部模型等待的 **25%**。其中 7 次是零思考、参数几百字，却等了 66–105 秒——最长那次参数只有 338 字，是一个发消息的调用。
工具执行占三分之一：中位 0.48 秒，但占一轮墙钟的 **33%**。最慢的 1% 吃掉其中 37%；WebFetch 中位 12 秒，6 次命令行卡在 120 秒超时。
跟上下文长度没关系：用 turn 内累计字符做代理，零思考调用的等待从 0.91 到 0.99 秒几乎不变，全量回归 r^2=0.01。**「预填充越来越长的上下文导致慢」没有数据支持。**

四、别的 harness 记什么

Claude Code 和 Codex CLI 都把这些数交给 OpenTelemetry，**默认都关着**。Codex 也按请求记 `ttft_ms`，但源码里**没有网关侧的响应头时刻**，这一段只有我们记。

五、已经能测的

首字延迟：Claude Code 每次请求的 `message_start` 本来就带 `ttft_ms`，平台此前没有一处读它。现在挂在每次调用写出的那条记录上，一次调用一个数。人为注入 1.0 秒和 0.4 秒延迟的两轮调用，读出的正是 1009 和 409 毫秒。
命令自身用时：执行器给每条命令计时，已验证（104 毫秒、67 秒两条真实记录）。Bash 工具那条路还没在会话记录里看到这个数，还差一步。
网关上游第一个字节：量网关到上游那一段，和 `ttft_ms` 相减能把「网络排队」和「预填充」分开。

六、还要做的

把命令用时接到 Bash 工具那条路上——差最后一步。
把已有记录 join 起来：上游部署 id、token 与缓存、轮次表三个时间点，数据都在，缺脚本。
沙箱镜像补工具链：`Dockerfile.private` 里只有 python3、bash、ca-certificates、ripgrep。一轮 21 次命令行调用里 10 次在补依赖，最后卡在共享库加载失败，**整轮零产出**。
读文件去重：同一份 `.typ` 在 4 轮里被读了 12 次。
平台提醒不占轮次：「状态 10 分钟没动过」阈值 600 秒，每句都开一整轮。

七、所以先做什么

第 1、2 条不改行为，能并行，做完每次调用等的每一段就都有数了。第 3–5 条也值得做，但都碰不到那 25% 的长尾和 33% 的工具时间。
长尾还没查清。等「排队」和「预填充」分开，再回头看那 7 次 66–105 秒。

口径：8 轮会话为 2026-11-01 下午；1,528 次调用为 10-23 至 11-01 全量。`ttft_ms` 实测于 Claude Code 2.1.299。